

DA2304 - Introduction to Computer Systems

Assignment 2

Kiran Kumar P
DA24B008

0. Extra Credit

My recent favorite song has been *Stalin Thodaratum Tamil Nadu Vellattum* by Arivalayam.

Link: [May 4: Wait and Watch](#)

1. Sanity Checking Logic

1.1 Why dimension validation is necessary?

For a matrix multiplication $W \times X$, we need $W_{\text{cols}} = X_{\text{rows}}$ to hold. If it doesn't, the inner product loop ends up iterating over the wrong number of elements. Since the Hack VM's address space is just a flat array of 16-bit integers with no type system or bounds checking, the loop will happily read whatever values happen to sit next to the matrix in RAM. You get garbage output and no runtime error to tell you something went wrong.

For B , we need $B_{\text{rows}} = W_{\text{rows}}$. If this doesn't match, the bias gets added to the wrong rows of the output - again, silent corruption with no way to detect it unless you check beforehand.

1.2 Validation Pseudocode

The convention I followed is: write -1 to `ERROR_LOC` on failure, 1 on success.

```
1 function sanity_check(W_rows, W_cols, X_rows, X_cols, B_rows,  
2   B_cols):  
3     // Check: matrix multiplication is defined  
4     if W_cols != X_rows:  
5         ERROR_LOC = -1  
6         return False  
7  
8     // Check: bias vector has matching row count  
9     if B_rows != W_rows:  
10        ERROR_LOC = -1
```

```

11         return False
12
13     // All dimensions are compatible
14     ERROR_LOC = 1
15     return True

```

Listing 1: Matrix dimension validation logic

For example, if W is 3×4 and X is 3×2 , then $W_{\text{cols}} = 4 \neq 3 = X_{\text{rows}}$. The check catches this right away and writes -1 to `ERROR_LOC` before any loop gets a chance to run.

1.3 Implementation in Hack VM

Push the two dimension arguments, subtract, and branch on the result. Here’s the first check:

```

1 // if W_cols != X_rows: error
2 push argument 1      // W_cols
3 push argument 2      // X_rows
4 sub
5 if-goto DIMENSION_ERROR
6
7 // ... rest of computation ...
8
9 label DIMENSION_ERROR
10 push constant 1
11 neg                // push -1
12 pop static 0       // ERROR_LOC = -1

```

Listing 2: VM-level dimension mismatch check

The key thing here is that `if-goto` fires on any nonzero value, so the subtraction result being nonzero means the dimensions don’t match.

2. Approach A: MATMUL Analysis

2.1 Architecture

In Approach A, one function `MATMUL(W, X, B, ...)` does the entire matrix multiplication by itself. It has a triple-nested loop that iterates over output rows, output columns, and the inner dot-product dimension. No helper functions, no calls - everything lives in one place.

2.2 Stack Usage Analysis

Since the entire computation runs inside a single `MATMUL` call, the stack frame is created only once. The return address and saved segment pointers (`LCL`, `ARG`, `THIS`, `THAT`) are pushed at the start and restored only at the end. All $m \times n \times k$ MAC operations happen inside this same frame without creating any extra call frames.

In practice, this keeps stack usage very stable. `SP` increases slightly when local variables are allocated, but after that it stays mostly flat throughout the nested loops. Since there

are no repeated `call/return` operations, the overhead from saving and restoring segment registers stays very low.

- Only one frame setup for the full computation.
- Minimal stack growth during execution.
- Much lower call overhead compared to ROWXCOL.
- Push/pop operations mainly come from arithmetic itself.

For larger matrices, this makes a noticeable difference because the frame setup cost stays fixed while the computation scales.

2.3 Segment Usage Analysis

Argument segment (ARG): The matrix pointers and dimensions are passed as arguments using ARG-relative addressing. In practice, it worked better to read these once at function entry and cache them into `temp` registers, so the repeated ARG access cost does not show up inside the inner loop.

Local segment (LCL): Loop counters like i , j , and k are stored as locals. Since local accesses go through LCL-relative addressing, they take more instructions compared to direct `temp` accesses. Because of that, I found it better to keep only the less frequently updated values in locals.

Temp segment (RAM[5-12]): This ended up being the most useful segment for the inner loop. Since `temp` maps directly to fixed RAM locations, accesses are much cheaper than going through LCL. While checking the generated assembly, the instruction count difference was pretty noticeable. Keeping the accumulator and pointer increments in `temp` reduced a lot of repeated overhead.

```
1 temp 0  --> dot product accumulator
2 temp 1  --> row pointer
3 temp 2  --> column pointer
4 local 0 --> i
5 local 1 --> j
6 local 2 --> k
```

Listing 3: Segment allocation for MATMUL

Static/Pointer segments: These were not really necessary for the core computation. Using `temp` for pointer handling was simpler and cleaner.

2.4 Advantages

- Only one stack frame for the entire computation.
- Row and column pointers can be reused efficiently.
- Much lower function-call overhead compared to ROWXCOL.
- Fewer instructions inside the inner loop.

2.5 Disadvantages

- **Large function body:** Hard to unit-test individual rows or columns in isolation.
- **High local variable count:** More locals mean a bigger frame footprint and more complex LCL-relative addressing.
- **Debugging difficulty:** While debugging, I noticed that a pointer arithmetic bug in the inner loop corrupts the entire output matrix at once, and there's no per-call boundary to help narrow down where things went wrong. An off-by-one in the column stride, for example, shifts every output element and it's not obvious which loop level caused it.

3. Approach B: ROWXCOL Analysis

3.1 Architecture

In Approach B, the caller handles the outer loops and delegates each dot product to a separate ROWXCOL(row_ptr, col_ptr, length) call. This function takes a pointer to a row of W , a pointer to a column of X , and the length of the vectors, and returns their dot product. The caller iterates over all (i, j) pairs, calls ROWXCOL for each one, adds the bias, and stores the result.

3.2 Stack Usage Analysis

ROWXCOL creates a brand-new stack frame for every single (i, j) output element. Each call pushes 3 arguments onto the stack, then the call mechanism pushes 5 more words (return address, saved LCL, ARG, THIS, THAT), for 8 words of stack growth per call. On return, all 8 get popped.

```
1 // Before call (caller pushes args):
2 Stack: [..., row_ptr, col_ptr, length]    // SP += 3
3
4 // During call (frame created):
5 Stack: [..., row_ptr, col_ptr, length,
6         ret_addr, saved_LCL, saved_ARG,
7         saved_THIS, saved_THAT]          // SP += 5 more
8
9 // Inside ROWXCOL (locals allocated):
10 Stack: [..., frame..., local_acc, local_k] // SP += num_locals
11
12 // On return: entire frame torn down
```

Listing 4: Stack state during one ROWXCOL call

For an $m \times n$ output, this whole sequence repeats $m \times n$ times. The cumulative stack traffic scales as $O(m \times n)$ frame operations, each costing roughly 30–50 assembly instructions for the call and a similar count for return. For a 4×4 output matrix, that's 16 call/return pairs, each saving and restoring 4 segment registers. The segment registers LCL, ARG, THIS, THAT get written to and read from the stack $16 \times 4 = 64$ times just in overhead - before any actual dot product computation even starts.

3.3 Segment Usage Analysis

Argument segment (ARG): ROWXCOL receives `row_ptr`, `col_ptr`, and `length` as arguments for every call. These are accessed using ARG-relative addressing (`push argument 0`, etc.). Unlike MATMUL, where arguments are usually cached once and reused, ROWXCOL ends up fetching them repeatedly. If the pointers are accessed directly inside the inner loop, the overhead grows quickly with every k iteration.

Local segment (LCL): The accumulator and loop counter are stored as locals. Since a fresh frame is created for every call, these locals are re-initialized each time. This keeps the function logic clean and isolated, but also means intermediate values cannot be reused across calls.

Temp segment: Caching frequently used values into `temp` made a noticeable difference during implementation. Since `temp` maps directly to fixed RAM addresses, accesses are cheaper than repeated ARG-relative fetches.

```
1 function ROWXCOL 2:
2     push argument 0
3     pop temp 0
4
5     push argument 1
6     pop temp 1
7
8     push argument 2
9     pop local 1
```

Listing 5: Caching arguments into temp inside ROWXCOL

Without this caching, the generated assembly becomes much larger than expected because of repeated `push argument` operations inside the loop.

Cross-call segment pressure: Every ROWXCOL call creates a new frame, so LCL and ARG keep getting saved and restored repeatedly. Compared to MATMUL, this creates much heavier stack and memory traffic.

3.4 Advantages

The biggest advantage of this approach is that ROWXCOL becomes a clean, reusable function. It can be tested independently using known vectors and reused for different matrix sizes without changing the logic. During debugging, this made things much easier since I could verify the dot product computation separately before checking the caller's loop or pointer updates.

3.5 Disadvantages

The downside is the function-call overhead. Every `call` and `return` expands into a fairly large amount of assembly for saving registers, setting up the frame, and restoring it later. Since ROWXCOL gets called for every output element, this overhead adds up quickly. Even for smaller matrices, the generated assembly had noticeably more call/return setup code than actual arithmetic instructions.

4. Comparison

Table 1: MATMUL vs. ROWXCOL: Architectural Comparison

Criterion	MATMUL (Approach A)	ROWXCOL (Approach B)
Stack frame setups	1 (for the entire computation)	$m \times n$ (one per dot product)
Argument passing overhead	Low (pointers loaded once from ARG)	High (3 arguments re-pushed every call)
Segment save/restore	Once on entry and exit	Once per ROWXCOL call
Execution efficiency	Better due to fewer redundant instructions	Worse due to repeated call overhead
Memory access pattern	Pointer arithmetic managed using temp segment	Pointers recomputed repeatedly for each call
Debugging	Harder to isolate failures	Easier to debug individual dot-product logic

MATMUL optimises for execution efficiency at the cost of structural clarity, while ROWXCOL prioritises correctness and testability at the cost of runtime performance.

5. Optimizations

5.1 Optimizations for MATMUL

Precomputing row offsets - Instead of recalculating `base + i * cols` every time, the row pointer can simply be incremented using a `temp` register. This avoids repeated multiplication logic, which is expensive on the Hack VM since multiplication itself is implemented through repeated addition.

Reducing unnecessary stack operations - A lot of instruction overhead came from values being pushed to the stack and immediately popped back. Keeping frequently used values like accumulators and pointers inside `temp` avoids this extra stack traffic.

Loop unrolling - If the inner dimension k is fixed and small, the loop can be manually unrolled to remove loop-counter updates and branch checks.

```

1 // Unrolled inner loop
2 push pointer 1
3 push pointer 2
4 add
5 // repeat manually

```

Listing 6: Manual loop unrolling for $k=4$

For smaller matrices, this noticeably reduces the instruction count.

Using the temp segment effectively - The `temp` segment maps directly to fixed RAM locations, so accesses are cheaper than `local` accesses through LCL. During implementation, storing the accumulator in `temp` instead of `local` reduced a surprising amount of repeated pointer-access overhead.

Hardware multiplier support - The Hack ALU does not support multiplication directly, so all multiplication logic becomes repeated addition. Even a simple hardware multiplier would reduce address computations and indexing overhead significantly.

5.2 Optimizations for ROWXCOL

Reducing frame setup overhead - Since ROWXCOL is called repeatedly, the save/restore overhead becomes expensive very quickly. A lighter calling convention for simple leaf functions could reduce unnecessary frame setup work.

Reusing locals efficiently - The accumulator and loop counter should be updated in-place instead of being repeatedly re-initialized. This cuts down unnecessary stack operations inside the loop.

Caching arguments in temp - Repeated ARG-relative accesses inside the inner loop increase instruction count quite a bit. Caching frequently used arguments like row and column pointers into `temp` registers at function entry made the generated assembly noticeably smaller and cleaner.